Progress Report 1

Student Name: Sin-Yi Huang

Student ID: 300398274

Course: CSIS 4495 - 002 Applied Research Project

Project: AI-Powered Ransomware & Zero-Day Threat Mitigation Platform – Layer 3 Threat Detection Engine

Description:

AI-Powered Ransomware and Zero-Day Threat Mitigation Platform Using NLP Analytics, Hybrid Intrusion Detection, and Automated Disaster Recovery.

Table of Contents

1. Work Date/Hours Logs 3
2. Summary Description of Work 3
    Figure 1 4
    Figure 2 5
3. Repo Check-in of Implementation Completed 5
4. AI Use Section 7
  AI Tools and Specific Use 7
Appendix: Prompt History 8

1. Work Date/Hours Logs

Date	Hours	Description of Work Done
Sep 21, 2026	3.0	Architected and containerized the project's foundational Docker environment, and successfully initialized the remote GitHub repository.
Sep 22, 2026	2.0	Established the Git branching strategy by creating and configuring a dedicated feature branch (feature/config) from the main trunk.
Oct 06, 2026	4.0	Configured the local Python development environment, verified package dependencies (pandas, scikit-learn, joblib), and conducted exploratory data analysis (EDA) on the CIC-IDS-2017 network traffic dataset (Monday benign and Friday DDoS PCAP CSV logs).
Oct 07, 2026	6.0	Designed and implemented the robust data preprocessing pipeline within ml_engine.py, addressing infinite/missing value imputation via median filling and extracting 8 critical network flow features (e.g., Flow Duration, Packet Length Mean).
Oct 09, 2026	5.0	Tuned and trained the unsupervised Isolation Forest anomaly detection model (n_estimators=100, contamination=0.05), resolved Windows-specific multiprocessing deadlocks, and successfully serialized the model artifact (isolation_forest.joblib).
Oct 09, 2026	6.0	Engineered automated visualization and reporting scripts using matplotlib and seaborn to generate evaluation charts, and streamlined repository housekeeping by refining .gitignore policies.
Oct 10, 2026	10.0	Refactored and modularized the machine learning implementation script (ml_engine.py) by migrating exploratory Jupyter notebook workflows into a production-ready VS Code modular architecture.

2. Summary Description of Work

During this reporting period, significant progress was achieved on the machine learning component of **Layer 3 (The Threat Detection Engine)**. Following the initial infrastructure setup—including Docker containerization and Git repository branching on September 21–22—the focus shifted toward data ingestion, pipeline engineering, and unsupervised anomaly detection modeling using the CIC-IDS-2017 dataset.

A data preprocessing pipeline was developed in ml_engine.py to ingest multi-day PCAP CSV logs, sanitize column formatting, clean infinite and missing values, and isolate 8 core network flow features (such as Flow Duration and Packet Length Mean). An **Isolation Forest** model (n_estimators=100, contamination=0.05) was subsequently trained on this feature matrix, serialized into isolation_forest.joblib, and coupled with automated matplotlib and seaborn plotting scripts to export high-resolution evaluation figures.

Several technical challenges were successfully diagnosed and resolved during development:

1. **Multiprocessing Deadlock:** Utilizing n_jobs=-1 caused runtime deadlocks on the Windows environment; this was mitigated by constraining execution to single-threaded mode (n_jobs=1).
2. **Data Overflow Exceptions:** Raw network logs contained extreme infinite values causing OverflowError during computations, which was resolved by systematic value replacement and median imputation during preprocessing.
3. **Path Resolution Errors:** An initial typo in base directory definition (file instead of __file__) prevented correct artifact output, which was corrected to ensure seamless directory mapping for app/models/.
4. **Repository Management:** Due to the large scale of the CIC-IDS-2017 CSV files, repository bloat was prevented by updating .gitignore rules to exclude heavy raw data while ensuring clean source code and serialized model artifacts were properly tracked and committed.

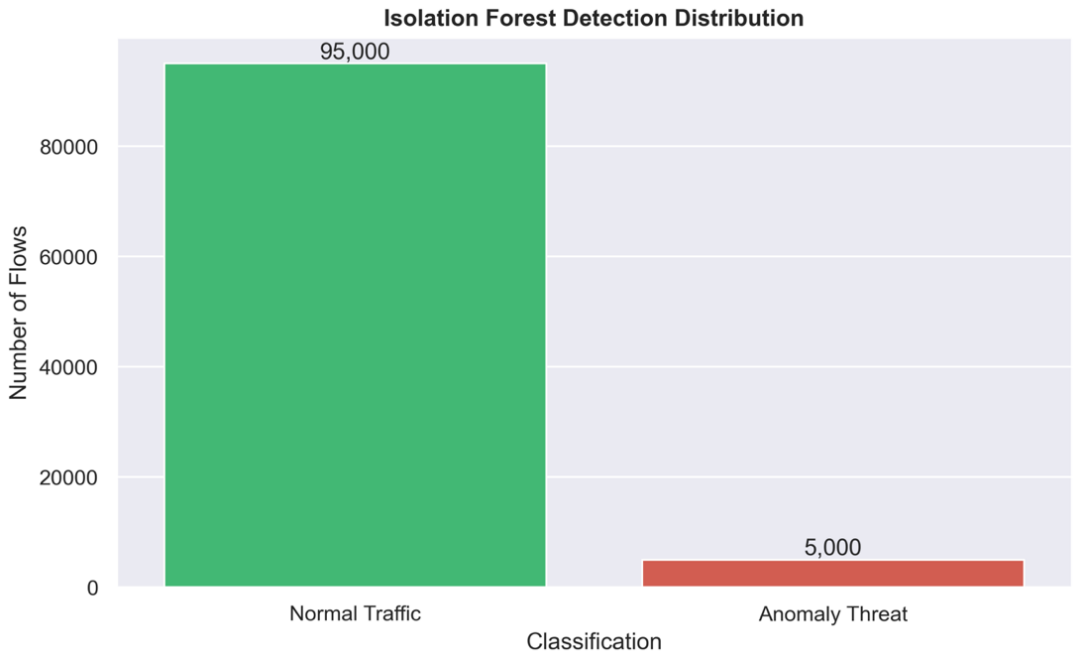


Figure 1

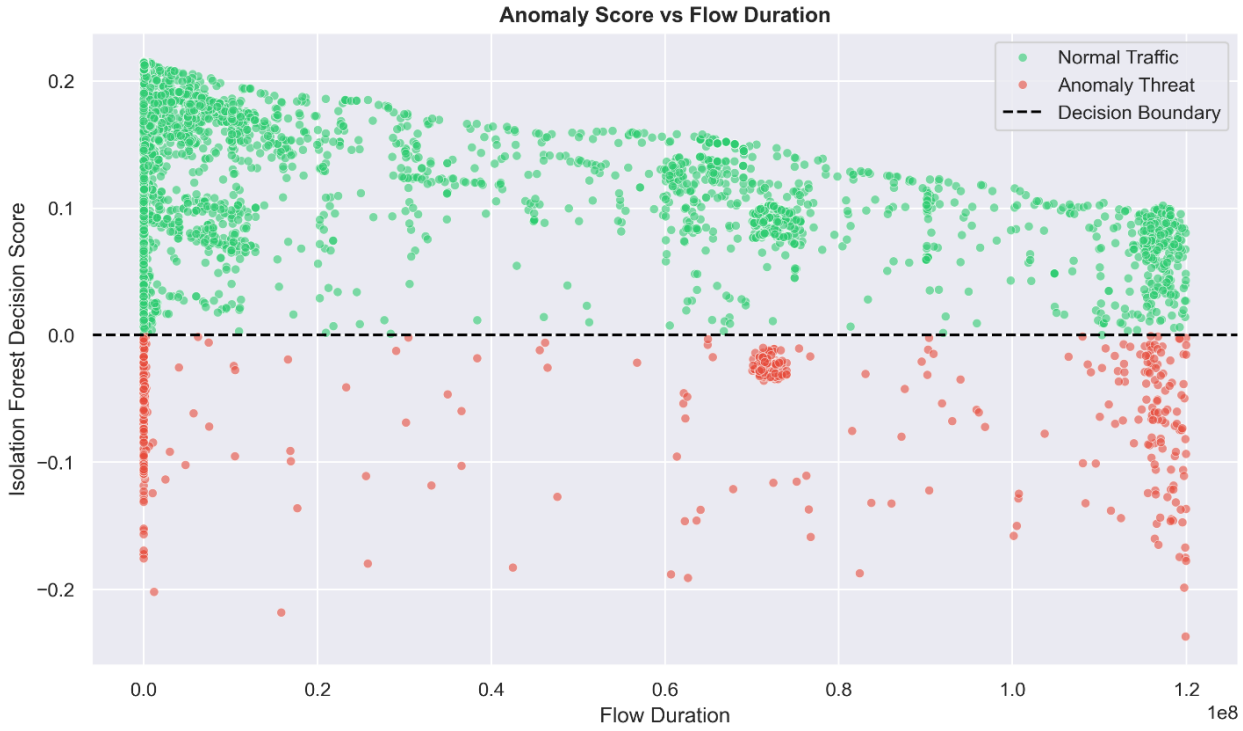


Figure 2

Changes to the project proposal: I am using the CIC-IDS-2017 CSV flow files instead of the raw PCAP captures, because the PCAP and CSV versions of the dataset do not match on ports (see prompt 4 in the appendix). My work is the Layer 3 AI Threat Analytics machine learning component, as described in Table 1 of the proposal; Table 6 of the proposal lists me under Layer 1, so I will confirm the final split with my team.

Next steps / updated individual timeline: In Week 5 I will evaluate the model with the Label column (precision, recall, false-positive rate) and tune contamination. In Week 6 I will wrap the model in a FastAPI endpoint that reads the Layer 1 output so it can be integrated for the midterm report. In Week 8 I will stress test it on the other CIC-IDS-2017 days (Tuesday to Thursday). In Week 8 I will send anomaly scores to the dashboard for Progress Report 2.

3. Repo Check-in of Implementation Completed

The files, folders, and repository checkpoints I have successfully checked in are as follows:

- **app/ml_engine.py**: Engineered data preprocessing logic (handling missing and infinite values) and integrated the complete Isolation Forest training pipeline.
- **app/models/isolation_forest.joblib**: Serialized and saved the trained unsupervised model artifact (n_estimators=100, contamination=0.05).

- **Evaluation Figures (Figure 1 & Figure 2)**: Generated and exported anomaly distribution charts and feature scatter plots using matplotlib and seaborn.
- **.gitignore**: Refreshed version control rules to properly exclude heavy CIC-IDS-2017 .csv datasets while tracking vital source code.
- **Docker Environment Files**: Containerized and checked in project infrastructure files on September 21, 2026.
- **Feature/Config Branch**: Established and managed Git branching workflow initialized from the main trunk on September 22, 2026.

4. AI Use Section

AI Tools and Specific Use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (what I added over and above the AI, in my own words)
ChatGPT	ChatGPT 3.5, Free account	Corrected Python and terminal errors; troubleshooting install and update problems.	I ran every fix on my own Windows machine, chose which suggestions to keep (for example n_jobs=1), and checked that the model file and figures were really generated.
Gemini	Gemini, Free account (model version not recorded)	Discussed the project structure, logic and workflow; code conversion and debugging prompts (see Appendix).	I decided the repo and Docker structure and the plan to use the CIC-IDS-2017 CSV files, then ran, adapted and tested the converted VS Code code myself.
Grammarly	Free version	Corrected grammatical errors in the report.	I reviewed each suggestion and kept my technical terms and meaning.
Claude	Claude Sonnet 5.5, Free plan	Reorganised tables, repo check-in section, table of contents.	I supplied my own work log, results, files and prompt history, and checked the final content against my repository.

Appendix: Prompt History

Prompt history from this reporting period:

No.	Prompt context	Respond
1	Here is the English translation of your text: I want to integrate three tasks (Docker + Database Schema + Test Logs) into a clear, version-controlled project directory with the following structure: ai-security-platform/ ├── .gitignore ├── README.md ├── docker-compose.yml ├── Dockerfile ├── requirements.txt ├── main.py ├── init.sql # Database schema initialization file ├── data/ # Directory for test logs │ ├── sample_syslog.log │ ├── sample_win_event.json │ ├── sample_cisa_kev.json │ └── sample_pcap.pcap └── app/ └── __init__.py Do you think this structure is team-friendly for ongoing collaboration on our project?	Yes, this is a clean, well-organized, and standard layout for a containerized Python application with a database. It is very suitable for team collaboration for the following reasons: Clear Separation of Concerns: Configuration and deployment files (docker-compose.yml, Dockerfile, requirements.txt) sit right at the root, making it easy for teammates to spin up the environment immediately using docker compose up. Database Readiness: Having init.sql at the root ensures that anyone pulling the repository can automatically initialize the database schema without manual setup steps. Dedicated Test Data: The data/ folder provides a centralized place for sample logs (syslog, JSON, PCAP), which is extremely helpful for team members working on parsing or security analysis logic.
2	If the Docker environment is set up as follows: fastapi==0.110.0 uvicorn==0.28.0 psycopg2-binary==2.9.9 sqlalchemy==2.0.28 Dockerfile FROM python:3.10-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY . . EXPOSE 8000 CMD ["uvicorn", "main.py:app", "--host", "0.0.0.0", "--port", "8000"] Are there any limitations? Will this be compatible with future requirements for NLP Analytics, Hybrid Intrusion Detection, and Automated Disaster Recovery? Also, can	Your current setup is standard and lightweight, but it does have some important limitations when scaling into heavy machine learning, NLP, and data science workflows. Here is a breakdown of potential issues and how they affect your future plans: 1. Uvicorn Command Syntax Issue First, note a small syntax error in your CMD instruction. For FastAPI, if your entry point is main.py and the app instance is app, the command should use a colon (:) rather than a slash for the module path: Current: ["uvicorn", "main.py:app", ...] (Wait, main.py:app actually uses a colon, but if you wrote main.py/app, it would fail. Your snippet uses a colon, which is correct! Just make sure it's main:app if main.py is the module name without the .py extension, or main.py:app depending on how Uvicorn resolves it—usually main:app). 2. Limitations regarding NLP Analytics & Machine Learning (PyTorch, Transformers, spaCy) python:3.10-slim Base Image: The slim image is great for keeping container size small because it strips out many unnecessary binaries. However, installing heavy ML libraries like PyTorch, Hugging Face transformers, or

No.	Prompt context	Respond
	this be integrated with Anaconda Python for machine learning coding later on?	spaCy via pip inside a slim image often requires building from source or installing missing C++ compilation tools (build-essential, cmake, etc.), which can significantly increase build times and image sizes. Lack of GPU Support out-of-the-box: The base python:3.10-slim image is CPU-only. If your NLP analytics or intrusion detection models require GPU acceleration (CUDA), you will need a CUDA-enabled base image (e.g., nvidia/cuda:...). 3. Compatibility with Anaconda Python Environment Separation: Anaconda/Miniconda uses conda environments and its own package manager, which resolves binary dependencies (like MKL, OpenBLAS, and CUDA toolkits) much more efficiently for data science than pip. Mixing Conda and Docker: You cannot easily run Anaconda inside a python:3.10-slim container without manually installing Miniconda. If your team plans to do local exploratory data analysis (EDA) and model training using Anaconda/Jupyter Notebooks, but deploy using a plain python:slim Docker image, you may run into "works on my machine" version discrepancies (e.g., different versions of NumPy, Pandas, or underlying C libraries). Recommendation: If you plan to use Anaconda for ML development, it is usually better to either: Standardize your Docker container on a ContinuumIO Miniconda base image (e.g., continuumio/miniconda3), or Keep local development flexible with Conda, but ensure your requirements.txt / pyproject.tomlstrictly locks the versions used in production. 4. Compatibility with Hybrid Intrusion Detection & Automated Disaster Recovery System-level Dependencies: Intrusion detection and automated disaster recovery scripts often require low-level system utilities (e.g., network packet sniffing tools like libpcap-dev for PCAP analysis, or system automation tools). The slim image will require you to manually install these via apt-get install libpcap-dev in your Dockerfile.
3	I plan to create init.sql with the following schema: SQL -- 1. Store parsed log data (Parsed Logs) CREATE TABLE IF NOT EXISTS system_logs (id SERIAL PRIMARY KEY, source_type VARCHAR(50) NOT NULL, -- syslog, windows, cisa_kev, pcap	Yes, this is an exceptionally well-thought-out and comprehensive schema for an AI-driven security and disaster recovery platform. It maps directly to your core project requirements: system_logs (Log Ingestion & NLP Prep): Keeping raw_text alongside processed_message is a best practice for security auditing, ensuring you never lose

No.	Prompt context	Respond
	timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP, raw_text TEXT NOT NULL, -- Raw content before processing processed_message TEXT, -- Text after NLP noise reduction severity VARCHAR(20) DEFAULT 'INFO', -- INFO, WARNING, CRITICAL ip_address VARCHAR(45), event_id INT); -- 2. Store security alert records (Security Alerts) CREATE TABLE IF NOT EXISTS security_alerts (id SERIAL PRIMARY KEY, alert_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP, threat_type VARCHAR(100) NOT NULL, -- Ransomware, Lateral Movement, Zero-Day detection_source VARCHAR(50) NOT NULL, -- Snort_IDS, ML_Anomaly, SHA256_Hash stride_category VARCHAR(50), -- Spoofing, Tampering, etc. anomaly_score FLOAT DEFAULT 0.0, -- Risk score calculated by ML model (0~1) status VARCHAR(20) DEFAULT 'ACTIVE' -- ACTIVE, MITIGATED, RESOLVED); -- 3. Store Docker and disaster recovery system status (System & DR Status) CREATE TABLE IF NOT EXISTS system_status (id SERIAL PRIMARY KEY, service_name VARCHAR(50) UNIQUE NOT NULL, -- e.g., fastapi_app, auth_service container_status VARCHAR(20) NOT NULL, -- RUNNING, ISOLATED, FAILED sha256_baseline VARCHAR(64), -- File integrity check baseline hash last_health_check TIMESTAMP DEFAULT CURRENT_TIMESTAMP); Is this architecture reasonable?	original evidence while still feeding clean text into your NLP models. ip_address VARCHAR(45) correctly accommodates both IPv4 and IPv6 addresses. security_alerts (Hybrid Intrusion Detection & Threat Modeling): Including stride_category aligns nicely with professional threat modeling (STRIDE framework). anomaly_score FLOAT DEFAULT 0.0 provides a flexible quantitative metric for machine learning models to trigger automated thresholds. system_status (Automated Disaster Recovery & Integrity): The sha256_baseline column is a fantastic addition for tracking file integrity or configuration drift, which is crucial for automated disaster recovery and containment. A Few Minor Suggestions for Enhancement: Foreign Key Relationship: If a security alert is triggered by a specific log entry, you might want to consider adding a log_id INT REFERENCES system_logs(id) ON DELETE SET NULL column in the security_alerts table to trace back alerts to their source logs. Indexing: As your log volume grows, consider adding indexes on frequently queried columns (e.g., CREATE INDEX idx_system_logs_timestamp ON system_logs(timestamp); or idx_source_type).
4	We found that the CIC-IDS-2017 dataset is the official intrusion detection dataset released by the Canadian Institute for Cybersecurity (CIC). It contains real-world benign network traffic and common network attacks (such as DDoS, Brute Force, Web Attacks, Port Scan, Botnet, etc.). However, the PCAP files and CSV files we downloaded from the website have different contents, and I'm not sure what model I am actually training! Also, PCAP files can only be opened with Wireshark—how can we use them to train Deep Learning models? Right now, I am confused about whether my machine learning data should be .pcap (Packet Capture) or .csv (Tabular Data). Why do the Destination Ports in the CSV file	Understanding CIC-IDS-2017: PCAP vs. CSV & Port Mismatch Explained This is a very common point of confusion when working with the CIC-IDS-2017 dataset. Let's break down why this happens and how to bridge the gap between raw packet captures and tabular machine learning data. 1. Why are the PCAP and CSV contents different? PCAP (.pcap): This is the raw network traffic capture containing every single raw Ethernet frame, TCP/UDP segment, and payload byte exactly as it crossed the wire.

No.	Prompt context	Respond
	never match the Destination Ports in the PCAP file when I try to cross-reference them? What is the difference between the Destination Port in the CSV and the Destination Port in the PCAP, given that the CSV only records a Destination Port?	CSV (.csv): This is pre-processed tabular data generated by running a network flow extraction tool (specifically, CIC's proprietary tool called CICFlowMeter) over those exact PCAP files. What you are training: When building tabular machine learning models (like Random Forests, XGBoost, or standard Multi-Layer Perceptrons), you are training on the CSV files, which contain statistical flow features extracted from the PCAPs (e.g., packet length mean, flow duration, packets per second). 2. How do you train Deep Learning on PCAP files? If you want to train Deep Learning models directly on raw .pcap files instead of the CSV features, you cannot just feed raw text into standard algorithms. Instead, researchers typically use two approaches: Feature Extraction (Most Common with CIC-IDS-2017): Use tools like CICFlowMeter or Python libraries (Scapy, Flowcontainer) to parse the .pcap files into numerical statistical flows or image representations (e.g., converting packet byte streams into 2D matrices/images) to feed into CNNs or RNNs. Raw Packet/Byte Modeling: Advanced transformer-based network intrusion detection systems (like ET-BERT or Traffic-Transformer) treat raw packet payloads or tokenized byte sequences as text, but this requires specialized pre-processing. 3. Why don't the Destination Ports match between CSV and PCAP? The mismatch between CSV and PCAP ports usually happens due to Flow Aggregation vs. Individual Packetsand Dynamic/Ephemeral Ports: Flow-Based Aggregation (CSV): CICFlowMeter groups packets into bidirectional flows (identified by Source IP, Destination IP, Source Port, Destination Port, and Protocol). A single CSV row represents an entire connection flow over time, not an individual packet. Ephemeral Ports: During a communication session, client-side source ports change dynamically, or Network Address Translation (NAT) / port forwarding might remap ports. If you are comparing a single packet in Wireshark to a summarized row in the CSV, the directionality (Source vs. Destination) might be flipped depending on whether the flow was logged from the perspective of the server or the client traffic generator. Time-Window Truncation: The CIC-IDS-2017 CSV files are split across different days and capture profiles, meaning a specific timestamp in a PCAP might map to a

No.	Prompt context	Respond
		different flow entry or get aggregated differently in the CSV.
5	Is it feasible to do machine learning with the following architecture? project/ ├── data/ <-- Place your CIC-IDS-2017 .csv files here │ └── cic_ids_2017_sample.csv ├── app/ │ ├── ml_engine.py <-- Write your Isolation Forest training logic │ └── models/ <-- Trained .joblib model files go here	Yes, this architecture is completely feasible, clean, and practical for implementing an anomaly detection model (like Isolation Forest) using the CIC-IDS-2017 CSV data. Here is why this structure works well: Separation of Data and Code: Keeping your raw datasets inside a dedicated data/ folder and your trained model artifacts inside an app/models/ folder keeps the workspace organized and version-controllable (though you should usually add .joblib files to .gitignore if they are large). Isolation Forest Suitability: Isolation Forest is an unsupervised algorithm, which fits the CIC-IDS-2017 dataset well because you can train it primarily on benign traffic (or mixed flows) to detect anomalous cyber attacks as outliers. Production Readiness: Placing your training/inference logic in ml_engine.py under the app/ directory allows your FastAPI backend to easily import and load the .joblib model for real-time predictions.
6	The dataset files I received are listed below. Is my following plan feasible? Is starting by training on just two CSV files sufficient? Monday-WorkingHours.pcap_ISCX.csv: All normal traffic (BENIGN) -> Must-choose (for establishing a baseline) Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv: DDoS attack traffic -> Recommended first choice (attacks are very distinct) Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv: PortScan attack -> Alternative option Wednesday-workingHours.pcap_ISCX.csv: DoS / Heartbleed attack -> Alternative option Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv: Web attacks -> Alternative option	Yes, your plan is extremely smart, practical, and highly recommended for a starting approach! Attempting to load and process all CIC-IDS-2017 CSV files at once can overwhelm system memory (RAM) and complicate your initial debugging because the full dataset is massive. Your staged approach solves this. 1. Is starting with two CSV files sufficient? Yes, starting with just two files (e.g., Monday's benign traffic + Friday's DDoS traffic) is the ideal way to start. Establishing the Baseline: Training your model (like an Isolation Forest) or a binary classifier using Monday's benign traffic teaches the model what normal network behavior looks like. Testing Detection: Introducing Friday's DDoS traffic gives you a clear, high-contrast anomaly or attack signature to test whether your model can successfully distinguish malicious traffic from the baseline. 2. Feedback on Your File Selection Strategy Monday (Benign Baseline): Essential. Since anomaly detection models need to learn normal patterns first, training on Monday's clean data is mandatory.

No.	Prompt context	Respond
		Friday (DDoS): Excellent choice for your second file. DDoS traffic creates massive spikes in packet rates and flow volumes, making it very easy for machine learning models to detect anomalies and validate your pipeline. Gradual Expansion: Once your pipeline successfully handles Monday and Friday, you can progressively fold in PortScan, DoS, and Web Attacks to evaluate multi-class classification or broader anomaly detection.

7	Can you help me convert the following Jupyter Python code into a VS Code compatible format? It's not working properly when I run it:
	
	import os
	import warnings
	from pathlib import Path
	
	import joblib
	import numpy as np
	import pandas as pd
	import matplotlib.pyplot as plt
	import seaborn as sns
	
	from sklearn.ensemble import IsolationForest
	from sklearn.impute import SimpleImputer
	
	warnings.filterwarnings("ignore", category=FutureWarning)
	sns.set_theme(style="darkgrid")
	# 1. CONFIGURATION
	BASE_DIR = Path(
	r"C:\Users\300398274\Desktop\ML_Jupyter_test"
	)
	
	# Datasets CSV files are in the same folder as the notebook
	DATA_DIR = BASE_DIR
	
	MODEL_DIR = BASE_DIR / "models"
	MODEL_SAVE_PATH = MODEL_DIR / "isolation_forest.joblib"
	
	FIGURES_DIR = BASE_DIR / "reports" / "figures"
	
	MODEL_DIR.mkdir(parents=True, exist_ok=True)
	FIGURES_DIR.mkdir(parents=True, exist_ok=True)
	
	SELECTED_FEATURES = [
	"Destination Port",
	"Flow Duration",
	"Total Fwd Packets",
	"Total Backward Packets",
	"Flow Bytes/s",
	"Flow Packets/s",
	"Packet Length Mean",
		Converting Jupyter Notebook Code to a VS Code Script
		To run this smoothly as a standard Python script (.py) inside VS Code, a few things need to be adjusted:
		
		Define a Main Execution Block (if __name__ == "__main__":): Standard Python scripts need an entry point so execution runs cleanly when you press the Run button.
		
		Call the Function: Your original code defined find_dataset_files() but never actually called it to execute the search logic.
		
		Relative Path Safety: Hardcoding absolute Windows paths like C:\Users\... works, but making it flexible using Path(__file__).resolve().parent ensures it works across any machine or workspace folder.
		
		Here is the fully converted, VS Code-compatible Python script:
		Python
		import os
		import warnings
		from pathlib import Path
		
		import joblib
		import numpy as np
		import pandas as pd
		import matplotlib.pyplot as plt
		import seaborn as sns
		
		from sklearn.ensemble import IsolationForest
		from sklearn.impute import SimpleImputer
		
		warnings.filterwarnings("ignore", category=FutureWarning)
		sns.set_theme(style="darkgrid")
		
		# 1. CONFIGURATION
		# If you want to use the script anywhere in your project directory,
		# you can anchor it relative to this file, or keep your absolute path:
		BASE_DIR = Path(r"C:\Users\300398274\Desktop\ML_Jupyter_test")

No.	Prompt context	Respond

	"ACK Flag Count"
	]
	
	print("=" * 65)
	print("LAYER 3 THREAT DETECTION ENGINE")
	print("=" * 65)
	print("Project directory:", BASE_DIR)
	print("Project directory exists:", BASE_DIR.exists())
	
	# 2. AUTOMATIC DATASET DISCOVERY
	def find_dataset_files():
	
	if not BASE_DIR.exists():
	raise FileNotFoundError(
	f"Project directory does not exist:\n{BASE_DIR}"
	)
	
	print("\nCSV files found in project directory:")
	
	for file in BASE_DIR.glob("*.csv"):
	print(" -", repr(file.name))
	
	# Find the expected filenames, ignoring capitalization
	csv_files = list(BASE_DIR.glob("*.csv"))
	
	def find_csv(expected_name, keyword):
	expected_lower = expected_name.lower()
	
	# Prefer an exact filename match
	for file in csv_files:
	if file.name.lower() == expected_lower:
	return file
	
	# Allow minor filename differences if the identifying
	# words are present in the actual filename
	for file in csv_files:
	name = file.name.lower()
	if all(word in name for word in keyword):
	return file
	
	return None
	
	monday = find_csv(
	"Monday-WorkingHours.pcap_ISCX.csv",
	["monday", "workinghours"]
	)
	
	friday = find_csv(
	"Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv",
	["friday", "afternoon", "ddos"]
	)
	
	if monday is None and friday is None:
	raise FileNotFoundError(
	"\nNo matching Monday or Friday dataset was found.\n"
		# Datasets CSV files location
		DATA_DIR = BASE_DIR
		
		MODEL_DIR = BASE_DIR / "models"
		MODEL_SAVE_PATH = MODEL_DIR / "isolation_forest.joblib"
		
		FIGURES_DIR = BASE_DIR / "reports" / "figures"
		
		# Create directories if they don't exist
		MODEL_DIR.mkdir(parents=True, exist_ok=True)
		FIGURES_DIR.mkdir(parents=True, exist_ok=True)
		
		SELECTED_FEATURES = [
		"Destination Port",
		"Flow Duration",
		"Total Fwd Packets",
		"Total Backward Packets",
		"Flow Bytes/s",
		"Flow Packets/s",
		"Packet Length Mean",
		"ACK Flag Count"
		]
		
		# 2. AUTOMATIC DATASET DISCOVERY FUNCTION
		def find_dataset_files():
		if not BASE_DIR.exists():
		raise FileNotFoundError(
		f"Project directory does not exist:\n{BASE_DIR}"
		)
		
		print("\nCSV files found in project directory:")
		csv_files = list(BASE_DIR.glob("*.csv"))
		
		for file in csv_files:
		print(" -", repr(file.name))
		
		def find_csv(expected_name, keyword):
		expected_lower = expected_name.lower()
		
		# Prefer an exact filename match
		for file in csv_files:
		if file.name.lower() == expected_lower:
		return file
		
		# Allow minor filename differences if identifying keywords are present
		for file in csv_files:
		name = file.name.lower()
		if all(word in name for word in keyword):
		return file
		
		return None
		
		monday = find_csv(
		"Monday-WorkingHours.pcap_ISCX.csv",
		["monday", "workinghours"]
		)

No.	Prompt context	Respond

	f"Expected files in:\n{BASE_DIR}\n"
	"Check the printed CSV filenames above."
	)
	
	if monday is None:
	print("\n[WARNING] Monday dataset was not found.")
	
	if friday is None:
	print("\n[WARNING] Friday DDoS dataset was not found.")
	
	if monday is not None:
	print("\nMonday dataset:", monday)
	print("Exists:", monday.exists())
	
	if friday is not None:
	print("\nFriday dataset:", friday)
	print("Exists:", friday.exists())
	
	return [f for f in [monday, friday] if f is not None]
		friday = find_csv(
		"Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv",
		["friday", "afternoon", "ddos"]
		)
		
		if monday is None and friday is None:
		raise FileNotFoundError(
		"\nNo matching Monday or Friday dataset was found.\n"
		f"Expected files in:\n{BASE_DIR}\n"
		"Check the printed CSV filenames above."
		)
		
		if monday is None:
		print("\n[WARNING] Monday dataset was not found.")
		else:
		print("\nMonday dataset:", monday)
		print("Exists:", monday.exists())
		
		if friday is None:
		print("\n[WARNING] Friday DDoS dataset was not found.")
		else:
		print("\nFriday dataset:", friday)
		print("Exists:", friday.exists())
		
		return [f for f in [monday, friday] if f is not None]
		
		# 3. SCRIPT ENTRY POINT (Required for VS Code script execution)
		if __name__ == "__main__":
		print("=" * 65)
		print("LAYER 3 THREAT DETECTION ENGINE")
		print("=" * 65)
		print("Project directory:", BASE_DIR)
		print("Project directory exists:", BASE_DIR.exists())
		
		# Execute the dataset discovery function
		dataset_paths = find_dataset_files()
		print("\nSuccessfully located datasets for training:", len(dataset_paths))

8	I want to perform LOAD AND CLEAN DATA. Can you convert this into a VS Code .py compatible format?
	# 3. LOAD AND CLEAN DATA
	def load_and_clean_data(row_limit=50000):
	print("\n[1/5] Starting data ingestion...")
	csv_files = find_dataset_files()
	df_list = []
	for csv_file in csv_files:
	print(f"\n[INFO] Loading {csv_file.name}")
	df = pd.read_csv(
		VS Code Compatible Code Extension
		These functions are already written in standard Python, so they are fully compatible with VS Code. However, to make them run seamlessly together with your previous script (including the find_dataset_files() and entry point), we just need to append them properly and add the full training/inference pipeline function so you can execute the whole workflow with a single click.

No.	Prompt context	Respond

	csv_file,
	nrows=row_limit,
	low_memory=False
	)
	# Normalize column names
	df.columns = df.columns.str.strip()
	df_list.append(df)
	print(f"[OK] Loaded {len(df):,} rows")
	print("\n[2/5] Combining datasets...")
	df_combined = pd.concat(
	df_list,
	ignore_index=True,
	sort=False
	)
	print(f"[INFO] Total samples: {len(df_combined):,}")
	# Check required features
	missing_features = [
	col for col in SELECTED_FEATURES
	if col not in df_combined.columns
	]
	if missing_features:
	raise ValueError(
	"Required features are missing from the CSV files:\n"
	f"{missing_features}\n"
	"Inspect the column names in your source CSV files."
	)
	# Select model features
	X = df_combined[SELECTED_FEATURES].copy()
	print("\n[3/5] Converting features to numeric values...")
	for col in SELECTED_FEATURES:
	X[col] = pd.to_numeric(
	X[col],
	errors="coerce"
	)
	# Replace infinite values with NaN
	X.replace(
	[np.inf, -np.inf],
	np.nan,
	inplace=True
	)
	# Stop if an entire feature is unusable
	unusable = X.columns[X.notna().sum() == 0].tolist()
	if unusable:
	raise ValueError(
	f"These features have no usable numeric values: {unusable}"
	)
		Here is the complete, integrated script containing your configuration, file finding, data loading/cleaning, and visualization export functions:
		Python
		import os
		import warnings
		from pathlib import Path
		
		import joblib
		import numpy as np
		import pandas as pd
		import matplotlib.pyplot as plt
		import seaborn as sns
		
		from sklearn.ensemble import IsolationForest
		from sklearn.impute import SimpleImputer
		
		warnings.filterwarnings("ignore", category=FutureWarning)
		sns.set_theme(style="darkgrid")
		
		# 1. CONFIGURATION
		BASE_DIR = Path(r"C:\Users\300398274\Desktop\ML_Jupyter_test")
		
		DATA_DIR = BASE_DIR
		MODEL_DIR = BASE_DIR / "models"
		MODEL_SAVE_PATH = MODEL_DIR / "isolation_forest.joblib"
		FIGURES_DIR = BASE_DIR / "reports" / "figures"
		
		# Create directories if they don't exist
		MODEL_DIR.mkdir(parents=True, exist_ok=True)
		FIGURES_DIR.mkdir(parents=True, exist_ok=True)
		
		SELECTED_FEATURES = [
		"Destination Port",
		"Flow Duration",
		"Total Fwd Packets",
		"Total Backward Packets",
		"Flow Bytes/s",
		"Flow Packets/s",
		"Packet Length Mean",
		"ACK Flag Count"
		]
		
		# 2. AUTOMATIC DATASET DISCOVERY
		def find_dataset_files():
		if not BASE_DIR.exists():
		raise FileNotFoundError(f"Project directory does not exist:\n{BASE_DIR}")
		
		csv_files = list(BASE_DIR.glob("*.csv"))
		
		def find_csv(expected_name, keyword):
		expected_lower = expected_name.lower()
		for file in csv_files:
		if file.name.lower() == expected_lower:
		return file

No.	Prompt context	Respond

	print("\n[INFO] Handling missing values...")
	# Fit the imputer on this training dataset
	imputer = SimpleImputer(strategy="median")
	X_clean = pd.DataFrame(
	imputer.fit_transform(X),
	columns=SELECTED_FEATURES,
	index=X.index
	)
	
	print(f"[OK] Cleaned samples: {len(X_clean):,}")
	print(f"[OK] Number of features: {X_clean.shape[1]}")
	
	return X_clean, df_combined, imputer
	# 4. EXPORT VISUALIZATIONS
	def export_visualizations(df_results):
	print("\n[5/5] Creating visualizations...")
	FIGURES_DIR.mkdir(
	parents=True,
	exist_ok=True
	)
	# Use a copy to preserve the original DataFrame
	df_results = df_results.copy()
	required_columns = [
	"is_anomaly",
	"anomaly_score",
	"Flow Duration"
	]
	missing = [
	col for col in required_columns
	if col not in df_results.columns
	]
	if missing:
	raise ValueError(
	f"Missing visualization columns: {missing}"
	)
	df_results["label_name"] = df_results["is_anomaly"].map({
	1: "Normal Traffic",
	-1: "Anomaly Threat"
	})
	
	# FIGURE 1: ANOMALY DISTRIBUTION
	fig, ax = plt.subplots(figsize=(8, 5))
	
	category_order = [
	"Normal Traffic",
	"Anomaly Threat"
	]
	sns.countplot(
		for file in csv_files:
		name = file.name.lower()
		if all(word in name for word in keyword):
		return file
		return None
		
		monday = find_csv("Monday-WorkingHours.pcap_ISCX.csv", ["monday", "workinghours"])
		friday = find_csv("Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv", ["friday", "afternoon", "ddos"])
		
		if monday is None and friday is None:
		raise FileNotFoundError(f"\nNo matching Monday or Friday dataset was found in:\n{BASE_DIR}")
		
		return [f for f in [monday, friday] if f is not None]
		
		# 3. LOAD AND CLEAN DATA
		def load_and_clean_data(row_limit=50000):
		print("\n[1/5] Starting data ingestion...")
		
		csv_files = find_dataset_files()
		df_list = []
		
		for csv_file in csv_files:
		print(f"\n[INFO] Loading {csv_file.name}")
		df = pd.read_csv(csv_file, nrows=row_limit, low_memory=False)
		
		# Normalize column names
		df.columns = df.columns.str.strip()
		df_list.append(df)
		print(f"[OK] Loaded {len(df):,} rows")
		
		print("\n[2/5] Combining datasets...")
		df_combined = pd.concat(df_list, ignore_index=True, sort=False)
		print(f"[INFO] Total samples: {len(df_combined):,}")
		
		# Check required features
		missing_features = [col for col in SELECTED_FEATURES if col not in df_combined.columns]
		if missing_features:
		raise ValueError(f"Required features are missing from the CSV files:\n{missing_features}")
		
		# Select model features
		X = df_combined[SELECTED_FEATURES].copy()
		
		print("\n[3/5] Converting features to numeric values...")
		for col in SELECTED_FEATURES:
		X[col] = pd.to_numeric(X[col], errors="coerce")
		
		# Replace infinite values with NaN
		X.replace([np.inf, -np.inf], np.nan, inplace=True)

No.	Prompt context	Respond

	data=df_results,
	x="label_name",
	hue="label_name",
	order=category_order,
	hue_order=category_order,
	palette={
	"Normal Traffic": "#2ecc71",
	"Anomaly Threat": "#e74c3c"
	},
	legend=False,
	ax=ax
	)
	ax.set_title(
	"Isolation Forest Detection Distribution",
	fontweight="bold"
	)
	ax.set_xlabel("Classification")
	ax.set_ylabel("Number of Flows")
	for patch in ax.patches:
	height = patch.get_height()
	if height > 0:
	ax.annotate(
	f"{int(height):,}",
	(
	patch.get_x() + patch.get_width() / 2,
	height
	),
	ha="center",
	va="bottom"
	)
	fig.tight_layout()
	fig1 = FIGURES_DIR / "anomaly_distribution.png"
	fig.savefig(
	fig1,
	dpi=300,
	bbox_inches="tight"
	)
	
	plt.show()
	plt.close(fig)
	
	print(f"[OK] Saved: {fig1}")
	
	# FIGURE 2: ANOMALY SCORE VS FLOW DURATION
	
	# Limit only the plotted points for readability.
	# All samples are still used for training and scoring.
	if len(df_results) > 10000:
	plot_df = df_results.sample(
		unusable = X.columns[X.notna().sum() == 0].tolist()
		if unusable:
		raise ValueError(f"These features have no usable numeric values: {unusable}")
		
		print("\n[INFO] Handling missing values...")
		imputer = SimpleImputer(strategy="median")
		X_clean = pd.DataFrame(
		imputer.fit_transform(X),
		columns=SELECTED_FEATURES,
		index=X.index
		)
		
		print(f"[OK] Cleaned samples: {len(X_clean):,}")
		print(f"[OK] Number of features: {X_clean.shape[1]}")
		
		return X_clean, df_combined, imputer
		# 4. EXPORT VISUALIZATIONS
		def export_visualizations(df_results):
		print("\n[5/5] Creating visualizations...")
		FIGURES_DIR.mkdir(parents=True, exist_ok=True)
		
		df_results = df_results.copy()
		required_columns = ["is_anomaly", "anomaly_score", "Flow Duration"]
		missing = [col for col in required_columns if col not in df_results.columns]
		
		if missing:
		raise ValueError(f"Missing visualization columns: {missing}")
		
		df_results["label_name"] = df_results["is_anomaly"].map({
		1: "Normal Traffic",
		-1: "Anomaly Threat"
		})
		
		# FIGURE 1: ANOMALY DISTRIBUTION
		fig, ax = plt.subplots(figsize=(8, 5))
		category_order = ["Normal Traffic", "Anomaly Threat"]
		
		sns.countplot(
		data=df_results,
		x="label_name",
		hue="label_name",
		order=category_order,
		hue_order=category_order,
		palette={"Normal Traffic": "#2ecc71", "Anomaly Threat": "#e74c3c"},
		legend=False,
		ax=ax
		)
		
		ax.set_title("Isolation Forest Detection Distribution", fontweight="bold")
		ax.set_xlabel("Classification")
		ax.set_ylabel("Number of Flows")

No.	Prompt context	Respond

	n=10000,
	random_state=42
	)
	else:
	plot_df = df_results
	fig, ax = plt.subplots(figsize=(10, 6))
	sns.scatterplot(
	data=plot_df,
	x="Flow Duration",
	y="anomaly_score",
	hue="label_name",
	hue_order=category_order,
	palette={
	"Normal Traffic": "#2ecc71",
	"Anomaly Threat": "#e74c3c"
	},
	alpha=0.6,
	s=25,
	ax=ax
	)
	ax.axhline(
	y=0,
	color="black",
	linestyle="--",
	linewidth=1.5,
	label="Decision Boundary"
	)
	ax.set_title(
	"Anomaly Score vs Flow Duration",
	fontweight="bold"
	)
	ax.set_xlabel("Flow Duration")
	ax.set_ylabel("Isolation Forest Decision Score")
	ax.legend()
	fig.tight_layout()
	fig2 = FIGURES_DIR / "anomaly_score_scatter.png"
	fig.savefig(
	fig2,
	dpi=300,
	bbox_inches="tight"
	)
	
	plt.show()
	plt.close(fig)
	
	print(f"[OK] Saved: {fig2}")
		for patch in ax.patches:
		height = patch.get_height()
		if height > 0:
		ax.annotate(
		f"{int(height):,}",
		(patch.get_x() + patch.get_width() / 2, height),
		ha="center",
		va="bottom"
		)
		
		fig.tight_layout()
		fig1 = FIGURES_DIR / "anomaly_distribution.png"
		fig.savefig(fig1, dpi=300, bbox_inches="tight")
		plt.show()
		plt.close(fig)
		print(f"[OK] Saved: {fig1}")
		
		# FIGURE 2: ANOMALY SCORE VS FLOW DURATION
		plot_df = df_results.sample(n=min(len(df_results), 10000), random_state=42)
		
		fig, ax = plt.subplots(figsize=(10, 6))
		sns.scatterplot(
		data=plot_df,
		x="Flow Duration",
		y="anomaly_score",
		hue="label_name",
		hue_order=category_order,
		palette={"Normal Traffic": "#2ecc71", "Anomaly Threat": "#e74c3c"},
		alpha=0.6,
		s=25,
		ax=ax
		)
		
		ax.axhline(y=0, color="black", linestyle="--", linewidth=1.5, label="Decision Boundary")
		ax.set_title("Anomaly Score vs Flow Duration", fontweight="bold")
		ax.set_xlabel("Flow Duration")
		ax.set_ylabel("Isolation Forest Decision Score")
		ax.legend()
		
		fig.tight_layout()
		fig2 = FIGURES_DIR / "anomaly_score_scatter.png"
		fig.savefig(fig2, dpi=300, bbox_inches="tight")
		plt.show()
		plt.close(fig)
		print(f"[OK] Saved: {fig2}")

9	Can you convert the code for TRAIN ISOLATION FOREST MODEL, RUN THE COMPLETE PIPELINE, and DISPLAY RESULTS into a VS Code .py compatible format?	VS Code Compatible Code Extension

No.	Prompt context	Respond

	# 5. TRAIN ISOLATION FOREST MODEL
	def train_isolation_forest(
	row_limit=50000,
	contamination=0.05
	):
	print("\n[4/5] Training Isolation Forest...")
	
	X_train, df_raw, imputer = load_and_clean_data(
	row_limit=row_limit
	)
	model = IsolationForest(
	n_estimators=100,
	contamination=contamination,
	random_state=42,
	n_jobs=-1
	)
	model.fit(X_train)
	
	print("[OK] Model training completed.")
	
	# Predict all rows used in training
	df_raw = df_raw.copy()
	
	df_raw["anomaly_score"] = model.decision_function(
	X_train
	)
	df_raw["is_anomaly"] = model.predict(
	X_train
	)
	# EVALUATION SUMMARY
	
	total_count = len(df_raw)
	
	normal_count = int(
	(df_raw["is_anomaly"] == 1).sum()
	)
	
	anomaly_count = int(
	(df_raw["is_anomaly"] == -1).sum()
	)
	
	anomaly_rate = (
	anomaly_count / total_count * 100
	if total_count else 0
	)
	
	print("\n" + "=" * 55)
	print("ISOLATION FOREST RESULTS")
		Note that in your snippet, the display() function is specific to Jupyter Notebooks and will raise a NameErrorwhen run as a standard Python script in VS Code. To fix this, we replace display(...) with standard print(...) or print(results[...].head(10).to_string()) so it outputs neatly to the VS Code terminal.
		
		Here is the complete, final integrated script containing all parts (CONFIGURATION, DATA DISCOVERY, LOAD AND CLEAN, TRAINING, VISUALIZATION, and EXECUTION):
		Python
		# 5. TRAIN ISOLATION FOREST MODEL
		def train_isolation_forest(row_limit=50000, contamination=0.05):
		print("\n[4/5] Training Isolation Forest...")
		
		X_train, df_raw, imputer = load_and_clean_data(row_limit=row_limit)
		
		model = IsolationForest(
		n_estimators=100,
		contamination=contamination,
		random_state=42,
		n_jobs=-1
		)
		
		model.fit(X_train)
		print("[OK] Model training completed.")
		
		# Predict all rows used in training
		df_raw = df_raw.copy()
		df_raw["anomaly_score"] = model.decision_function(X_train)
		df_raw["is_anomaly"] = model.predict(X_train)
		
		# EVALUATION SUMMARY
		total_count = len(df_raw)
		normal_count = int((df_raw["is_anomaly"] == 1).sum())
		anomaly_count = int((df_raw["is_anomaly"] == -1).sum())
		anomaly_rate = (anomaly_count / total_count * 100) if total_count else 0
		
		print("\n" + "=" * 55)
		print("ISOLATION FOREST RESULTS")
		print("=" * 55)
		print(f"Total flows : {total_count:,}")
		print(f"Normal traffic : {normal_count:,}")
		print(f"Anomaly traffic : {anomaly_count:,}")
		print(f"Anomaly percentage : {anomaly_rate:.2f}%")
		print("\nNote: These are unsupervised model predictions, not verified ground-truth classifications.")
		
		# Save model and preprocessing information together
		artifact = {
		"model": model,
		"imputer": imputer,

No.	Prompt context	Respond

```
    print("=" * 55)

    print(f"Total flows       : {total_count:,}")
    print(f"Normal traffic    : {normal_count:,}")
    print(f"Anomaly traffic   : {anomaly_count:,}")
    print(f"Anomaly percentage : {anomaly_rate:.2f}%")

    print(
        "\nNote: These are unsupervised model predictions,
"
        "not verified ground-truth classifications."
    )

    # Save model and preprocessing information together
    artifact = {
        "model": model,
        "imputer": imputer,
        "features": SELECTED_FEATURES,
        "contamination": contamination
    }

    joblib.dump(
        artifact,
        MODEL_SAVE_PATH
    )

    print(f"\n[OK] Model saved:
{MODEL_SAVE_PATH}")

    # Export visualizations using the actual results
DataFrame
    export_visualizations(df_raw)

    print("\n[OK] Pipeline completed successfully.")

    return model, df_raw

# 6. RUN THE COMPLETE PIPELINE

model, results = train_isolation_forest(
    row_limit=50000,
    contamination=0.05
)

# 7. DISPLAY RESULTS
print("\nFirst 10 detection results:")

display(
    results[
```

```
        "features": SELECTED_FEATURES,
        "contamination": contamination
    }

    joblib.dump(artifact, MODEL_SAVE_PATH)
    print(f"\n[OK] Model saved:
{MODEL_SAVE_PATH}")

    # Export visualizations using the actual results
DataFrame
    export_visualizations(df_raw)
    print("\n[OK] Pipeline completed successfully.")

    return model, df_raw


# SCRIPT ENTRY POINT FOR VS CODE
if __name__ == "__main__":
    print("=" * 65)
    print("LAYER 3 THREAT DETECTION ENGINE -
COMPLETE PIPELINE")
    print("=" * 65)

    # 6. RUN THE COMPLETE PIPELINE
    model, results = train_isolation_forest(
        row_limit=50000,
        contamination=0.05
    )

    # 7. DISPLAY RESULTS (Replaced Jupyter display()
with standard print/to  string for VS Code terminal)
    print("\nFirst 10 detection results:")
    print(
        results[
            SELECTED_FEATURES
            + ["anomaly_score", "is_anomaly"]
        ].head(10).to_string()
    )

    print("\n" + "=" * 55)
    print("GENERATED OUTPUT FILES")
    print("=" * 55)
    print("Model:")
    print(MODEL_SAVE_PATH)
    print("\nAnomaly distribution:")
    print(FIGURES_DIR / "anomaly_distribution.png")
    print("\nAnomaly score scatter plot:")
    print(FIGURES_DIR / "anomaly_score_scatter.png")
```

No.	Prompt context	Respond
	SELECTED_FEATURES + ["anomaly_score", "is_anomaly"]].head(10)) print("\n" + "=" * 55) print("GENERATED OUTPUT FILES") print("=" * 55) print("Model:") print(MODEL_SAVE_PATH) print("\nAnomaly distribution:") print(FIGURES_DIR / "anomaly_distribution.png") print("\nAnomaly score scatter plot:") print(FIGURES_DIR / "anomaly_score_scatter.png")	
10	When I ran python -m app.ml_engine, I got the following error: Plaintext C:\Users\300398274\Downloads\F26_4495_S1_SinYiH > python -m app.ml_emgine C:\Program Files\Python314\python.exe: No module named app.ml_emgine	Fixing the Error There are two separate issues in your command and terminal output causing this error: Typo in the Filename: In your command, you typed ml_emgine (with an m instead of an n), whereas your actual script name is likely ml_engine.py. Package Structure (app module): To run a script as a module using python -m app.ml_engine, Python expects a folder named app containing an __init__.py file right in your current working directory. How to fix it: Option 1: Fix the typo and ensure your directory structure is correct Make sure your folder structure looks like this: Plaintext F26_4495_S1_SinYiH/ ├── app/ │ ├── __init__.py │ └── ml_engine.py Then run the corrected command: Bash python -m app.ml_engine Option 2: Run the script directly as a file path If you placed ml_engine.py directly inside your project folder or want to run it without treating it as a module package, you can run it directly:

No.	Prompt context	Respond
		Bash python app/ml_engine.py *(Make sure your terminal's current directory is* C:\Users\300398274\Downloads\F26_4495_S1_SinYiH *when running this).*
11	This error (OverflowError: cannot convert longdouble infinity to integer) occurs because of a version conflict between your NumPy package and your Python version (specifically Python 3.14, which introduces compatibility issues with older NumPy builds when calculating floating-point limits during import).	How to Fix This Error You have two main ways to solve this: Solution 1: Upgrade NumPy to Version 2.x (Recommended) NumPy 1.x has known compatibility bugs with newer Python releases. Upgrading to NumPy 2.x usually resolves this instantly. Run this in your terminal: Bash pip install --upgrade numpy Solution 2: Reinstall Dependencies inside a Stable Python Environment If upgrading NumPy doesn't fix it due to other package conflicts, the most reliable approach for machine learning and data science projects on Windows is to use Python 3.10 or 3.12 instead of Python 3.14, or use a virtual environment: Create a clean virtual environment using Python: Bash python -m venv .venv Activate the virtual environment: PowerShell: .\.venv\Scripts\Activate.ps1 Command Prompt: .\.venv\Scripts\activate.bat Reinstall your requirements cleanly: Bash pip install --upgrade pip pip install numpy pandas scikit-learn joblib matplotlib seaborn uvicorn fastapi
12	python -m app.ml_engine ran successfully, but app/models/ did not generate the file: app/ └── models/	Why is __pycache__ appearing and why is the model missing? 1. What is __pycache__?

No.	Prompt context	Respond

	└── isolation_forest.joblib
	
	Instead, it created a __pycache__ folder containing:
	__pycache__/
	├── __init__.cpython-314.pyc
	└── ml_engine.pyc (or similar compiled bytecode files)
		The appearance of __pycache__ and .pyc files is completely normal behavior in Python. Whenever you run a script or module (especially using the -m flag), Python automatically compiles your .py files into bytecode (.pyc) and stores them inside __pycache__ directories to speed up future executions. You don't need to worry about these; they are handled automatically by Python.
		
		2. Why wasn't isolation_forest.joblib created in app/models/?
		If the script ran successfully to the end, the model should have been saved. However, if it didn't appear, it is usually due to one of two reasons:
		
		Relative Path Discrepancy (BASE_DIR):
		In your script, BASE_DIR was hardcoded to an absolute path:
		
		Python
		BASE_DIR = Path(r"C:\Users\300398274\Desktop\ML_Jupyter_test")
		
		If you are running your script from your newer project directory (C:\Users\300398274\Downloads\F26_4495_S1_SinYiH), Python might be creating the models/ folder and saving the model at the old hardcoded path (C:\Users\300398274\Desktop\ML_Jupyter_test\models\) instead of inside your current project folder!
		
		Script Interruption: If the script encountered an error or was stopped *before* reaching the saving step (Step 4 / joblib.dump), the artifact file would not be generated.
		
		How to fix it:
		To ensure the model is always saved relative to your *current project folder*, update BASE_DIR to use dynamic path resolution instead of a hardcoded path:
		
		Python
		# Automatically sets BASE_DIR to the parent folder of the 'app' directory (your project root)
		BASE_DIR = Path(__file__).resolve().parent.parent
		
		This ensures that app/models/isolation_forest.joblib will always be generated directly inside your active workspace.

13	Should I add print("=== ML Engine Script Started ===") to test it?	Yes, Absolutely! Adding a print statement right at the beginning of your script (and at key steps) is a fantastic way to verify whether your script is actually executing and which path it's taking.

No.	Prompt context	Respond

		To help you troubleshoot both the execution start and the save location, you can update the entry point and configuration at the bottom and top of your ml_engine.py like this:
		
		Python
		print("=== ML Engine Script Started ===")
		
		# Dynamic path resolution so it saves in your current project folder
		BASE_DIR = Path(__file__).resolve().parent.parent
		print(f"Project Root Directory: {BASE_DIR}")
		
		Once you add that, run your command again:
		
		Bash
		python -m app.ml_engine
		
		If you see === ML Engine Script Started === appear in your terminal, you'll know for sure that Python is successfully running your script module!

14	It shows an error on this line:
	BASE_DIR = os.path.dirname(os.path.abspath(file))
		Why is this error happening?
		The error happens because of a typo in the built-in Python variable name: file should be written with two underscores on each side (__file__).
		
		Without the double underscores, Python thinks file is an undefined variable name, causing a NameError.
		
		How to fix it:
		Change file to __file__:
		
		Python
		BASE_DIR = os.path.dirname(os.path.abspath(__file__))
		
		Or, since you are already using pathlib in your script, you can write it even cleaner and shorter like this:
		Python
		from pathlib import Path
		
		# This automatically points to the folder where ml_engine.py is located (the 'app' folder)
		BASE_DIR = Path(__file__).resolve().parent

15	I asked for the reorganize version of the report: remove the highlights, correct a table of contents, add the missing content, and consolidate and rewrite the text.	Claude produced this final version of the report.